Low-latency messaging with Aeron

Microseconds latency, still extremely high throughput

Agenda

- Introduction to Aeron
- Benchmark
- Why is it fast ?
- Is it reliable ?
- Trade-off

Aeron introduction

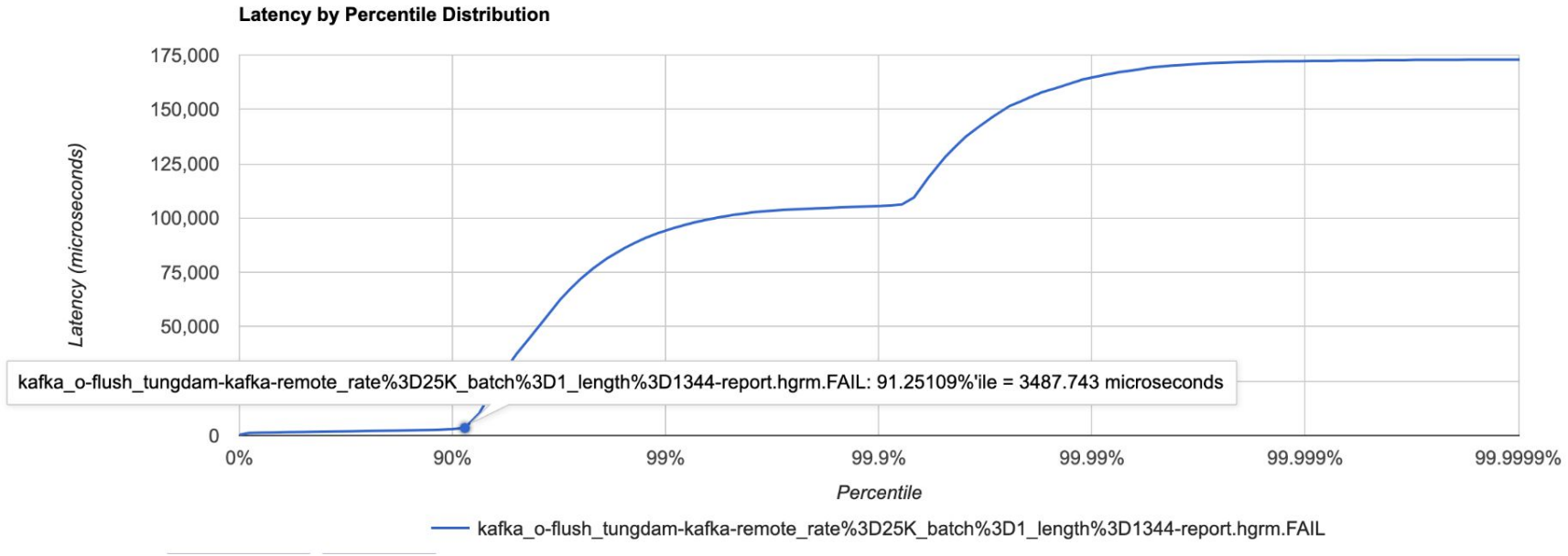
(Adaptive) Aeron introduction

- Global technology standard for high-throughput low-latency, fault-tolerant messaging solution in trading systems
- Open source with premium feature / support
- High performance message transport
- Asynchronous persistent log. Record and replay
- Fault-tolerant with RAFT
- Microsecond latency for “trading at the speed of light”

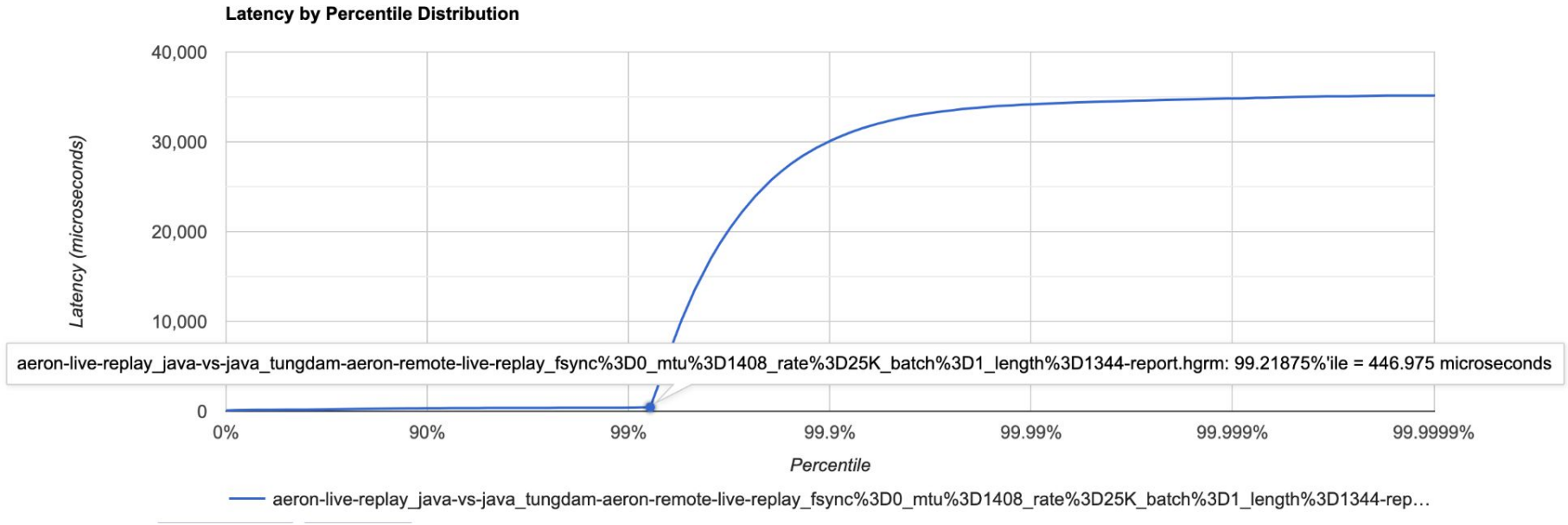
Benchmark

- Local benchmark using aeron-io/benchmarks code
  - 2 Linux machines over 1Gbps cable
  - Server node: Core i5 1240P, 32GB DDR4, NVME SSD
  - Kafka 3.9.1 - single node setup
  - Aeron 1.47 - single node Aeron transport + archive
  - 90 iterations (warm up + real test) of 25k msg batch
  - 1344 bytes (data + header) each
  - Notes:
    - The benchmark is limited my machines network cards
    - Aeron is not a drop-in replacement of kafka
- More benchmarks from Adaptive on GCP and AWS: < 250 us p99 for ~ 250k msg per sec (inside AZ)

Kafka


Latency by Percentile Distribution
175,000
150,000
125,000
100,000
75,000
50,000
0
Latency (microseconds)
kafka_o-flush_tungdam-kafka-remote_rate%3D25K_batch%3D1_length%3D1344-report.hgrm.FAIL: 91.25109%'ile = 3487.743 microseconds
0%
90%
99%
99.9%
99.99%
99.999%
99.9999%
Percentile
kafka_o-flush_tungdam-kafka-remote_rate%3D25K_batch%3D1_length%3D1344-report.hgrm.FAIL

Aeron


Latency by Percentile Distribution
40,000
30,000
20,000
10,000
0
Latency (microseconds)
aeron-live-replay_java-vs-java_tungdam-aeron-remote-live-replay_fsync%3D0_mtu%3D1408_rate%3D25K_batch%3D1_length%3D1344-report.hgrm: 99.21875%'ile = 446.975 microseconds
0%
90%
99%
99.9%
99.99%
99.999%
99.9999%
Percentile
aeron-live-replay_java-vs-java_tungdam-aeron-remote-live-replay_fsync%3D0_mtu%3D1408_rate%3D25K_batch%3D1_length%3D1344-rep…

Why is it fast ?

“Performance is the key focus”: clear design principle

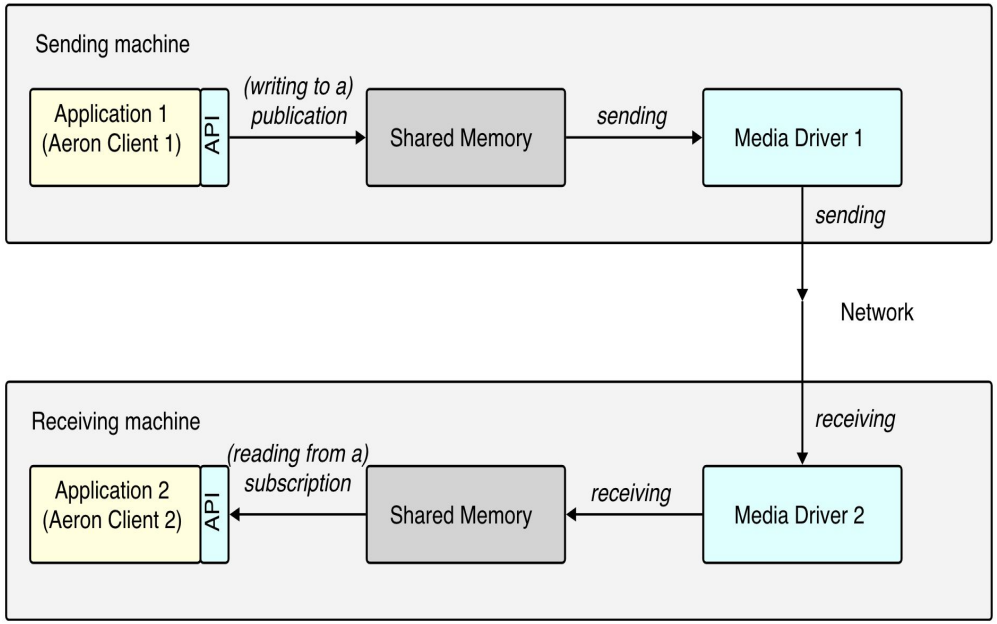
- Garbage free in steady state running
- Apply smart batching in the message path
- Lock-free algorithms in the message path
- Non-blocking IO in the message path
- No exceptional case in message path
- Apply single writer principle
- Prefer unshared state
- Avoid unnecessary data copies

“Performance is the key focus”: multi-layer optimization

- Agent-based architecture to leverage the multi-everything modern hardware
- First ever session-based flow-control UDP-based transport protocol (L4 + L5)
- Simple Binary Encoding for high performance message codec (L6)
- Agrona high performance data structure and utilities to overcome existing limitation of existing ones in Java (which proved to serve no GC design principle very well) (L7)

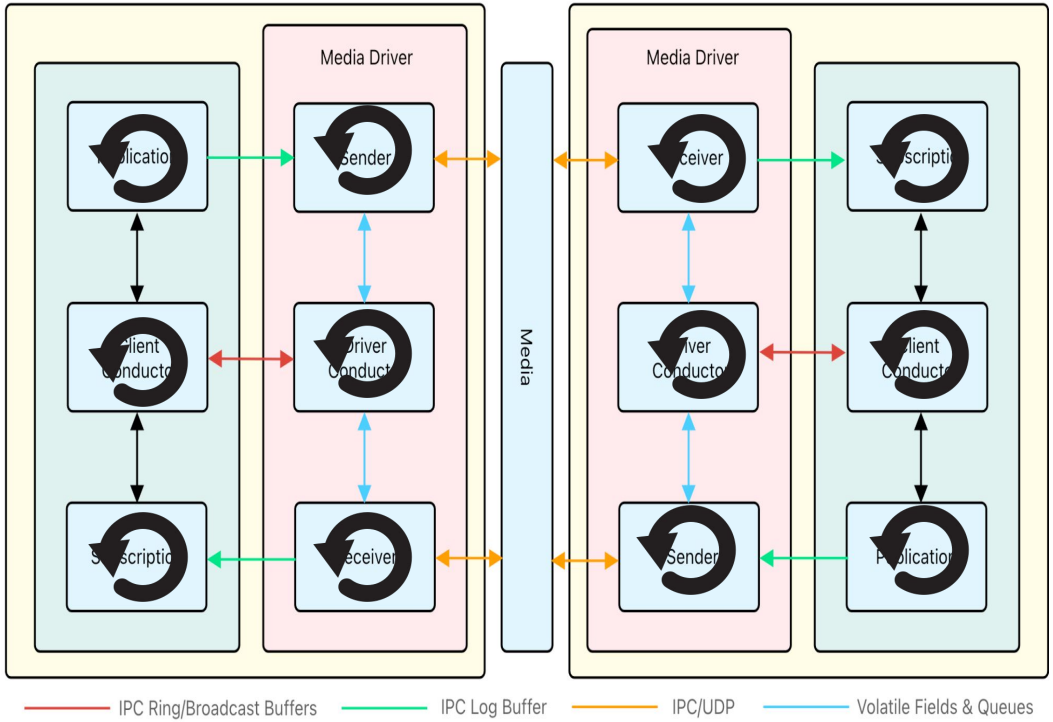
Aeron transport

- Brokerless, peer to peer messaging
- A library instead of framework
- “Sidecar” media driver
- Inter-process communication or Reliable UDP (multi/unicast) with flow control



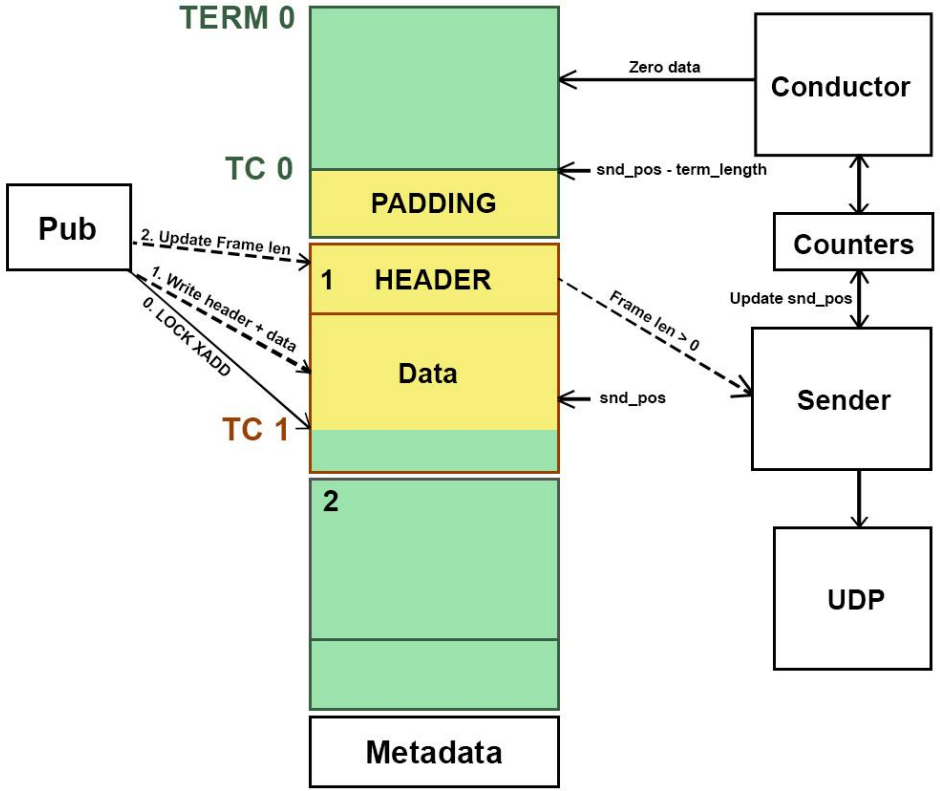
Aeron transport (cont)

- Pipeline vs monolithic
- Aggressive, CPU-cache friendly event-loop / duty-cycle
- Fast IPC queue is the key



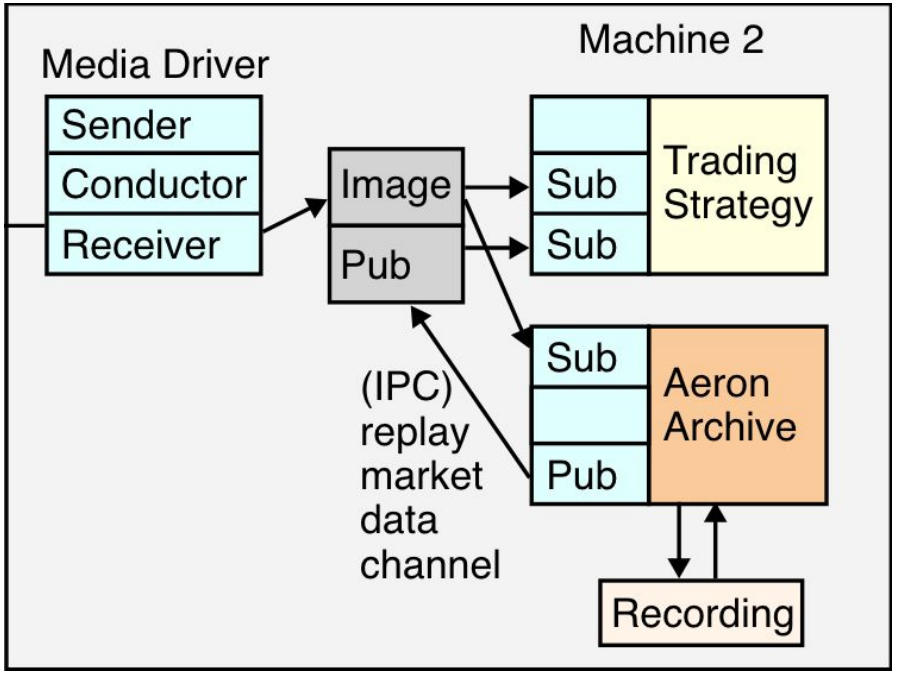
The queue - logbuffer

- A file but in shared memory (/dev/shm)
- Mmaped to share between processes and avoid copy
- Ring buffer with 3 different “terms” for easy background zeroing process
- Publishers race to bump a tail-counter to reserve place only
- Data writing process is wait-free

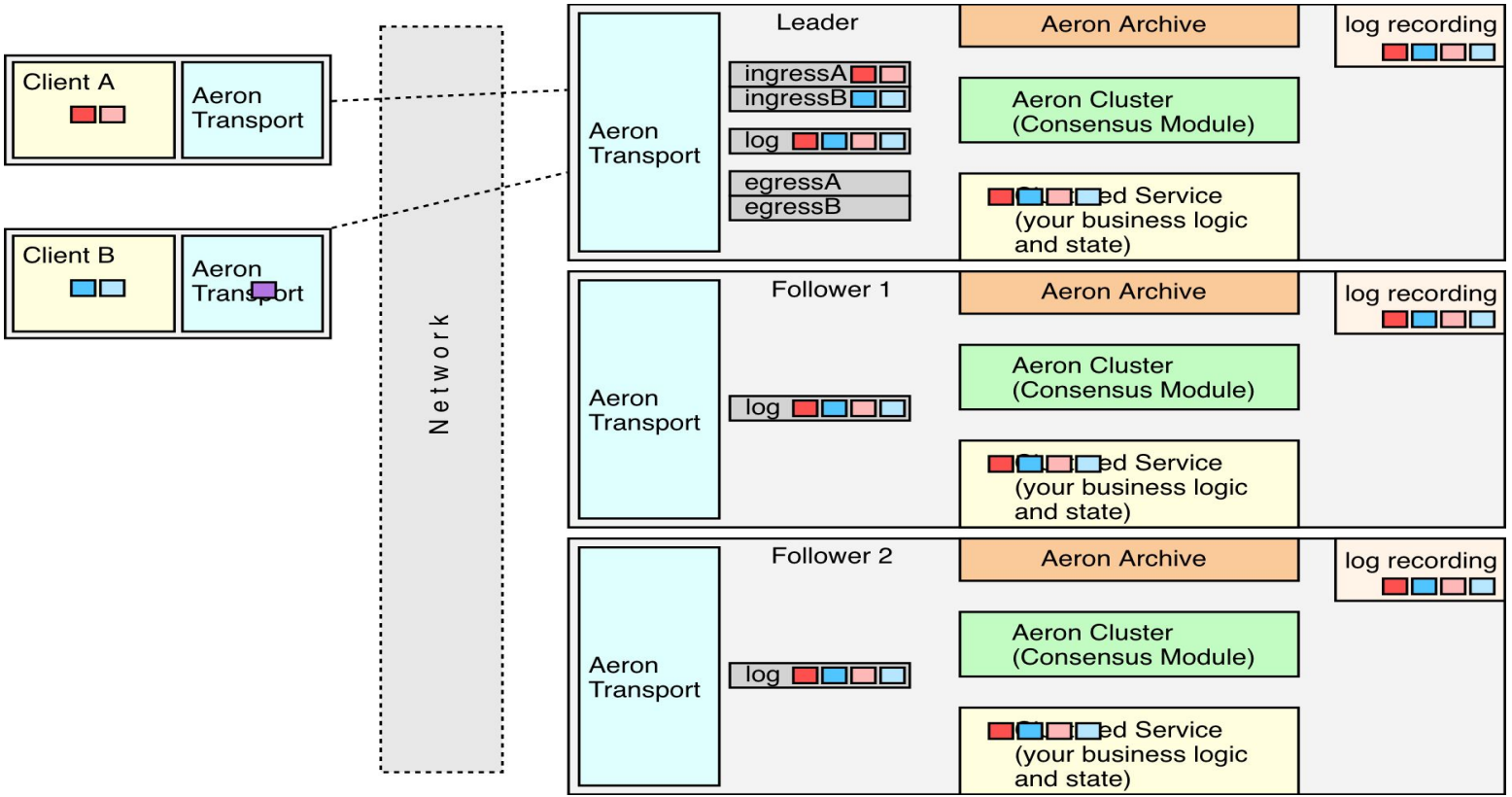


Aeron archive

- Record and replay
- Ensure durability asynchronously
- Works on either sending or receiving end
- Built on top of aeron transport



Architecture - Aeron cluster



Is it reliable?

Aeron reliability feature

- Durable, persistence and fast recovery by snapshot / replay with Aeron Archive.
- Reliable transport: delivery assurance: loss detection by subscriber's image counters, NAK and retransmit
- Partition tolerance and High Availability with Aeron cluster: RAFT consensus on top of Aeron Transport and Archive

Backup, recovery and failover

- Snapshot and replay from Archive log
- Aeron backup
- Aeron standby (premium feature)

Limitations

- No topic like kafka - only 1 single replicated state machine persistent log
- Development cost: difficult to follow agent-based, duty-cycle, unidirectional flows characteristics
- Only support C++ and Java client officially (It’s enough ?)
- Relatively resource greedy

Reference

- Aeron official docs
- Aeron github wiki
- Better than official docs
- "Aeron: Open-source high-performance messaging" by Martin Thompson
- Adventures with concurrent programming in Java: A quest for predictable latency by Martin Thompson
- Faster or better design ? choose any two
- Single Writer principle
- A concurrency cost hierarchy by Travis Down (RedPanda maintainer)

THANK YOU !